



Modal

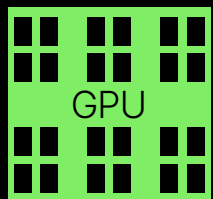
Achieve Truly Serverless GPUs

with libfuse,
checkpoint/restore,
& cuda-checkpoint

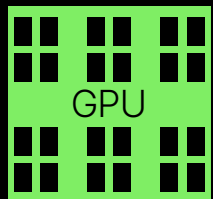
Why serverless GPUs?

Maximize GPU *Allocation* Utilization
and match supply to demand.

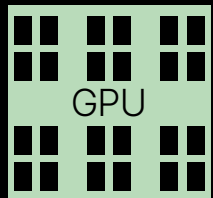
We're familiar with GPU *Kernel* Utilization.



100%

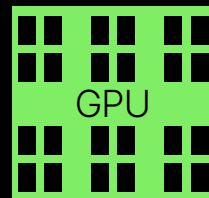
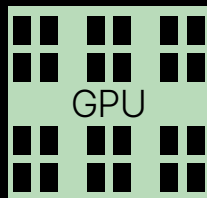
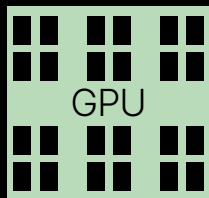
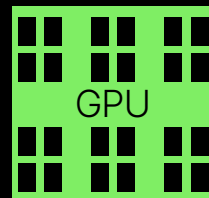
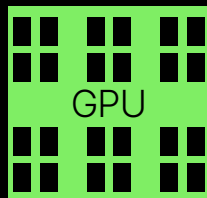
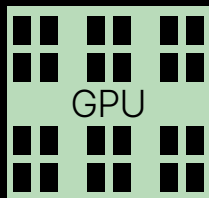
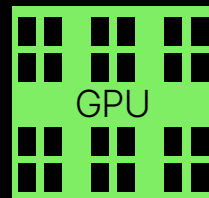
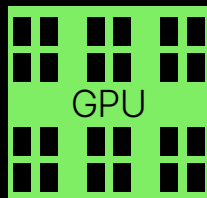
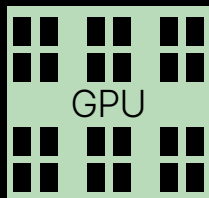


30%



0%

GPU *Allocation* Utilization is higher-level, and trickier!

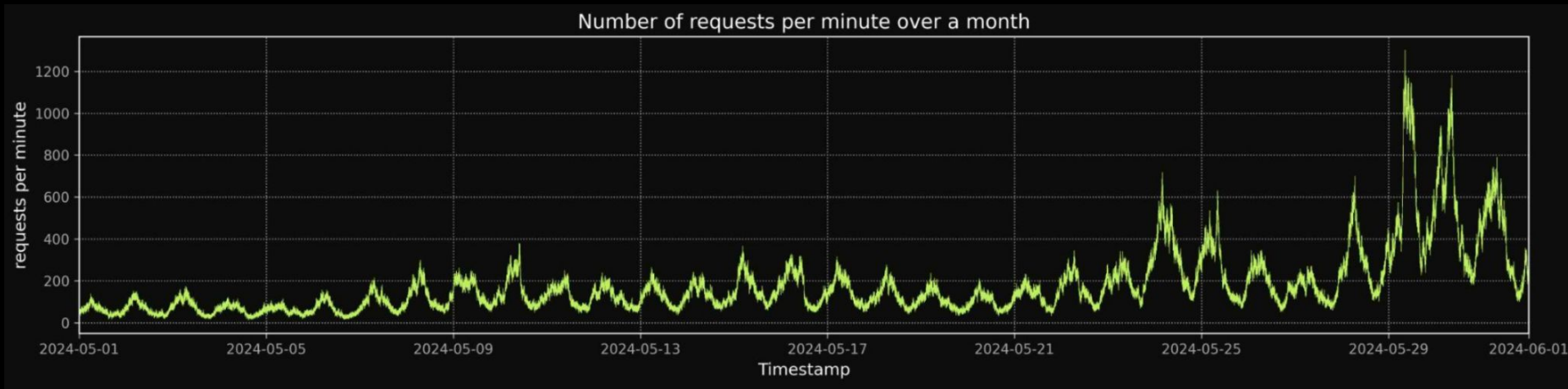


0%

67%

100%

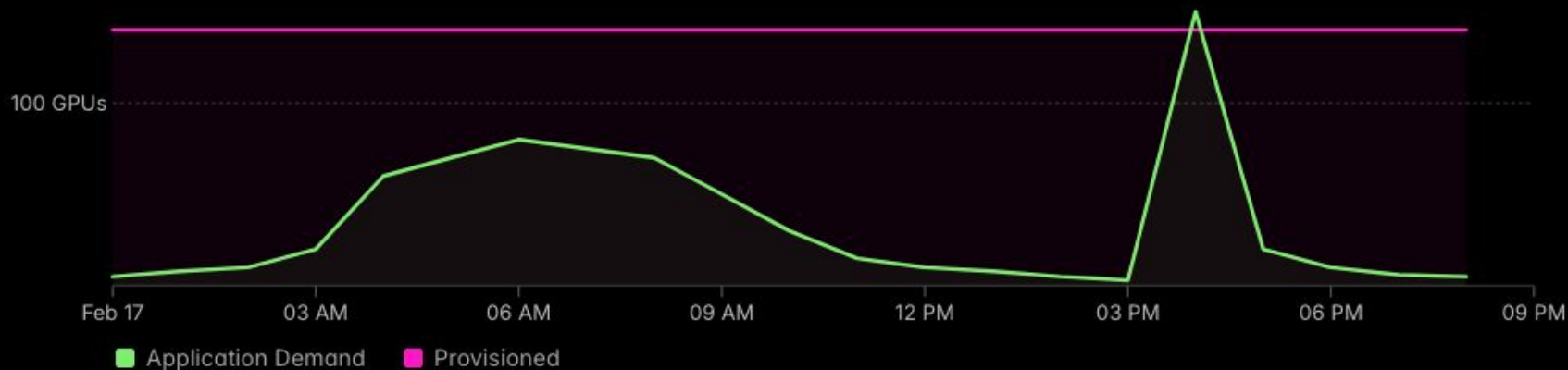
Inference applications have variable demand.



Engineers must efficiently service that demand within SLO.

Fixed, over-provisioned GPU allocations are under-utilized.

With a fixed, over-provisioned GPU allocation, utilization is low



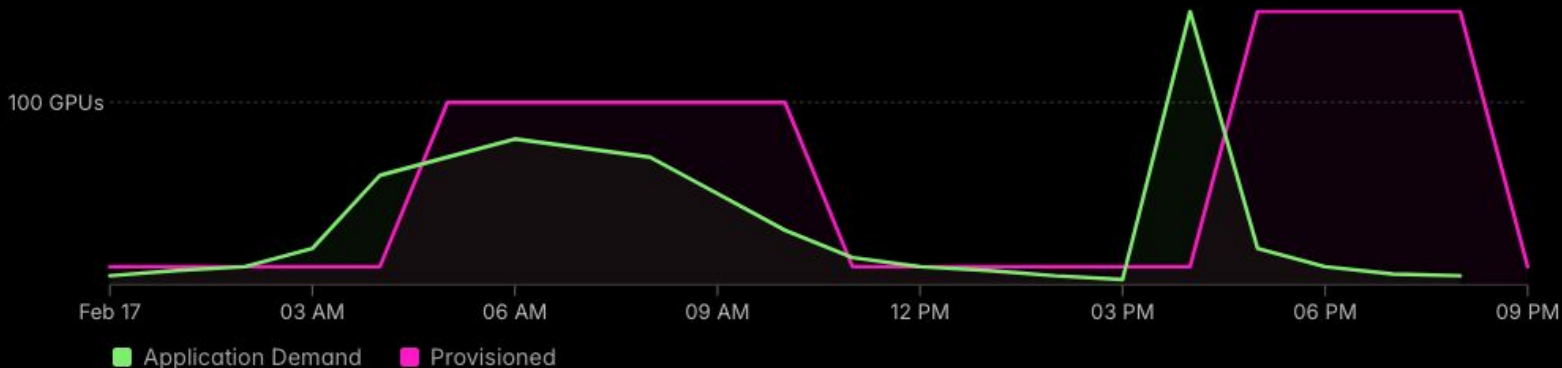
'I paid for the whole GPU, I am going to use the whole GPU': A high-level guide to GPU utilization

modal.com/blog/gpu-utilization-guide



Automatic allocation does *not* fix this – if it is slow.

If allocation is slow, utilization and QoS suffer



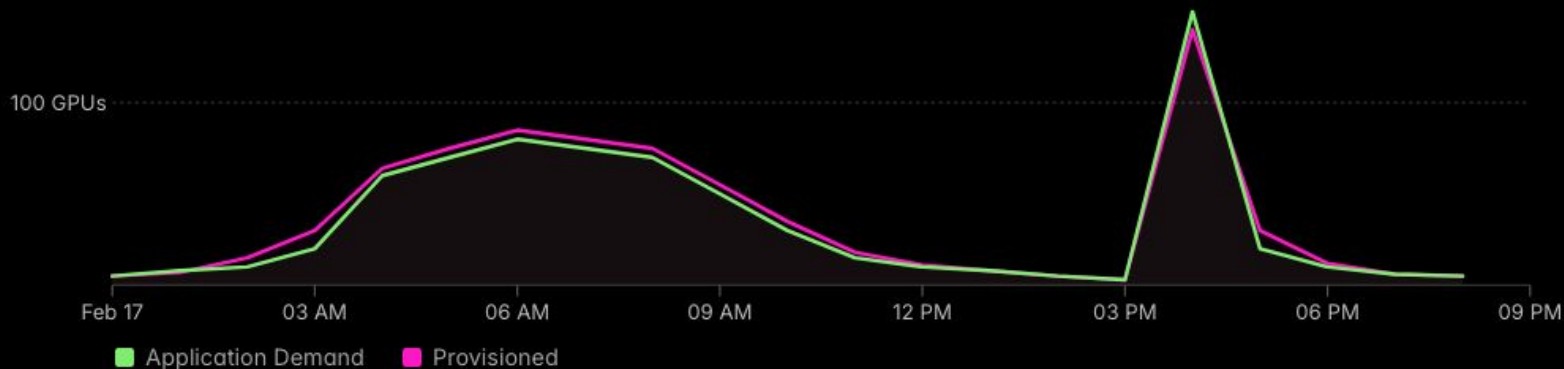
'I paid for the whole GPU, I am going to use the whole GPU': A high-level guide to GPU utilization

modal.com/blog/gpu-utilization-guide



Automatic allocation must be fast to handle variable demand.

With fast, automatic allocation, utilization and QoS can both be high

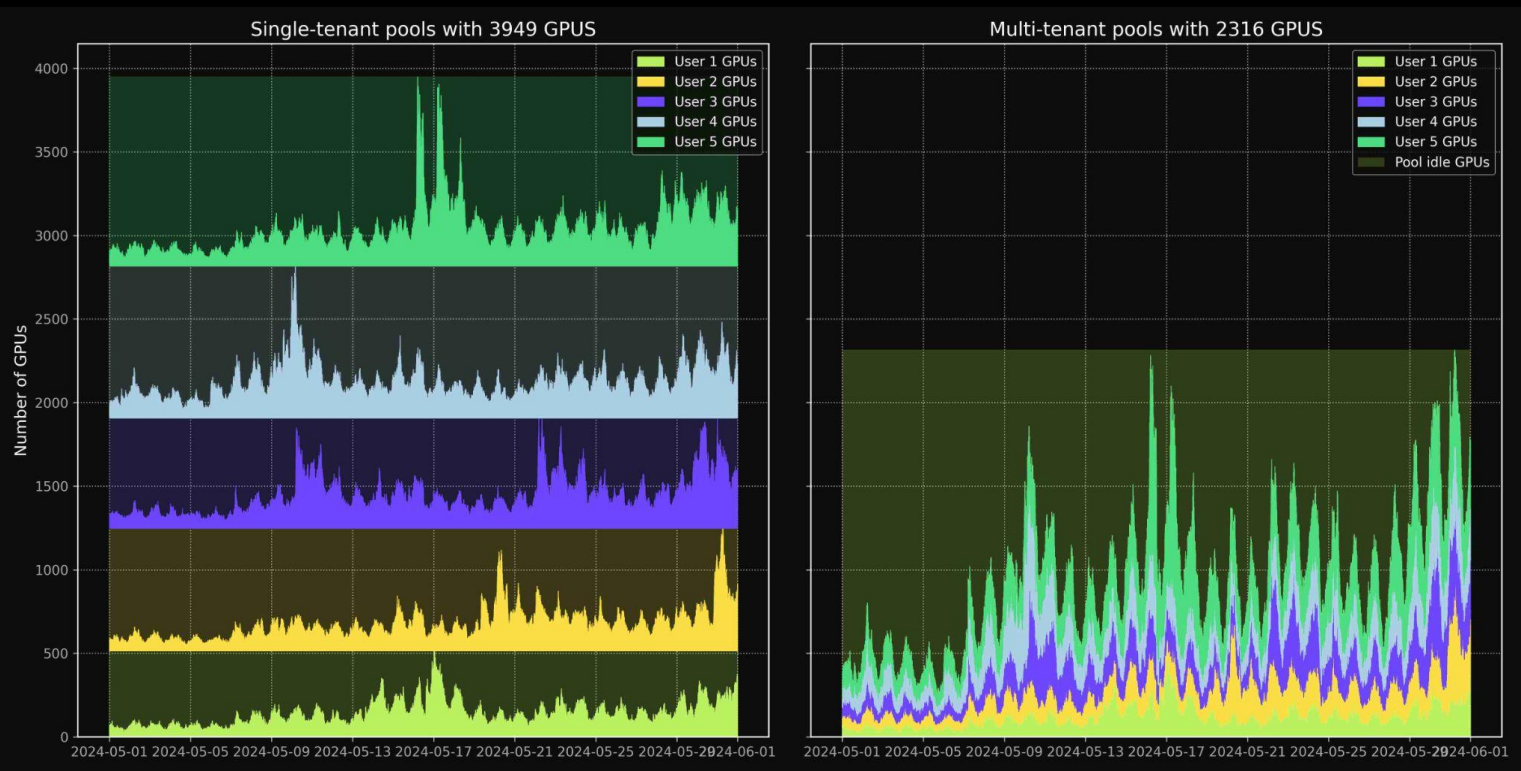


'I paid for the whole GPU, I am going to use the whole GPU': A high-level guide to GPU utilization

modal.com/blog/gpu-utilization-guide



Fast automatic allocation also unlocks better demand pooling.



Modal: a global compute platform designed for high GAU.

$\sim 10^4$
GPUs

*typically running
concurrently*

$\sim 10^9$
Sandboxes

total executed

$\sim 10^1$
Clouds

*providing
capacity*



scale

ramp 

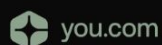
Quora



substack

Harvey

Meta



Lovable



SUNO



Mistral AI



Cartesia

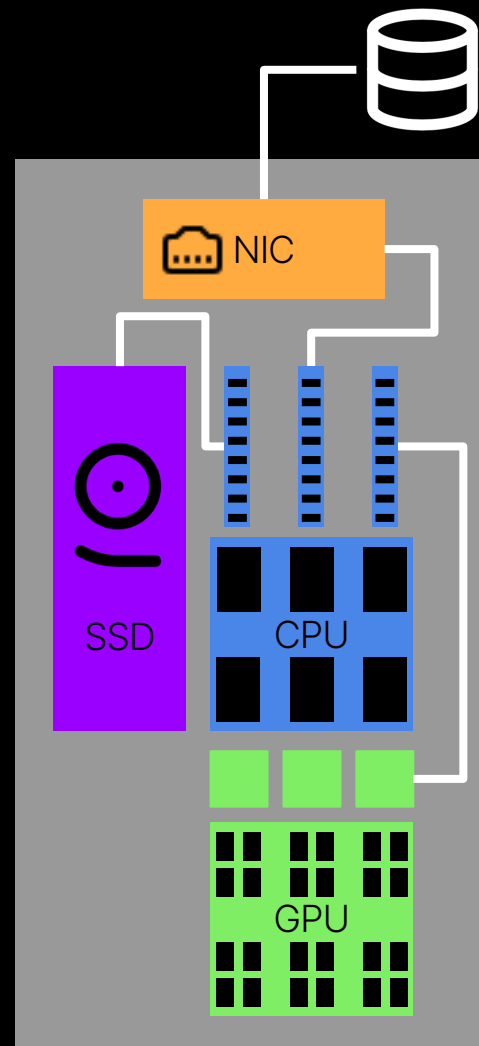
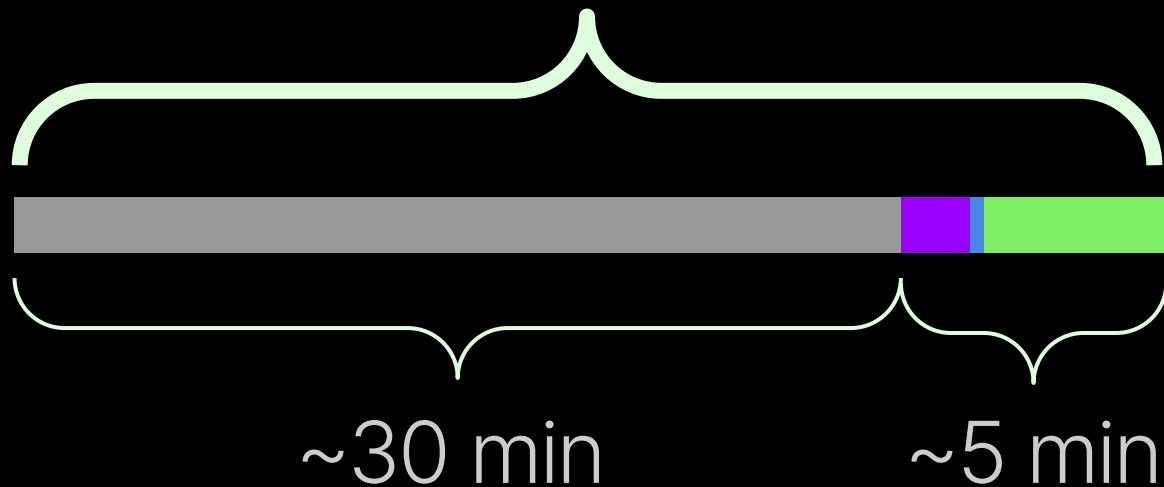
Trusted by leading AI teams

How we optimized startup latency

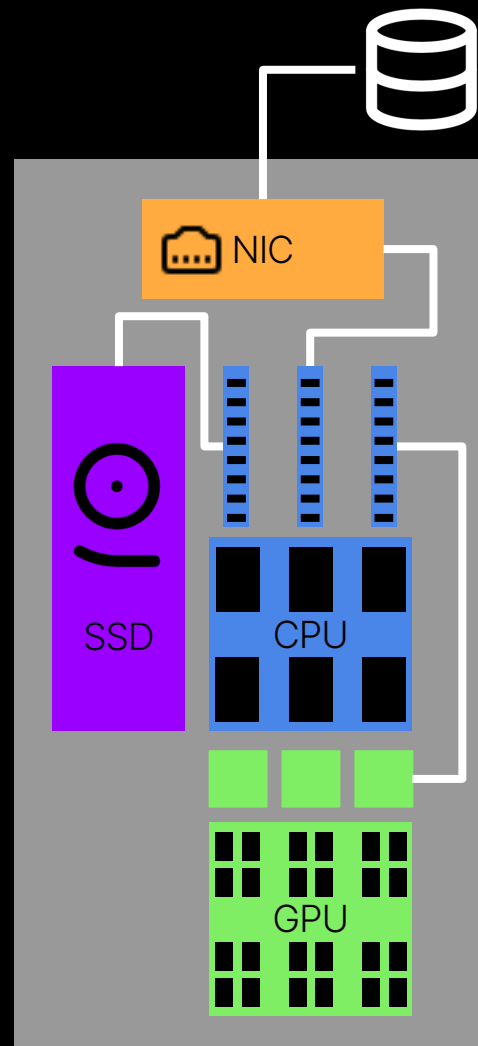
*Don't do it.
Do it, but don't do it again.
Do it when they're not looking.*

- Hanson & Crain

~2 ksec



$\sim 1/20$ ksec



Spin up a new instance

Eagerly load filesystem

Start application on host

Start application on device

Maintain a buffer

Lazy-load filesystem

Restore Linux process

Restore CUDA ctx

Spin up a new instance

Eagerly load filesystem

Start application on host

Start application on device

Maintain a buffer

Lazy-load filesystem

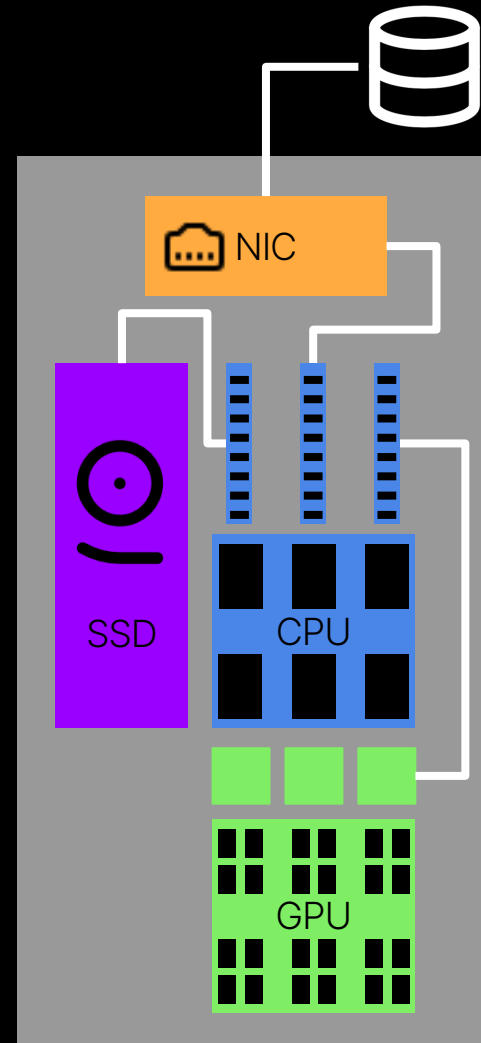
Restore Linux process

Restore CUDA ctx

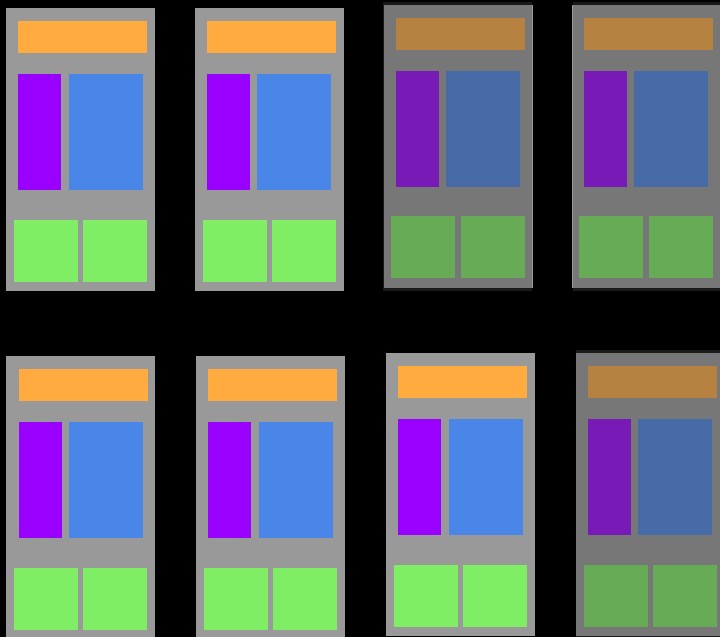
What happens *here*?



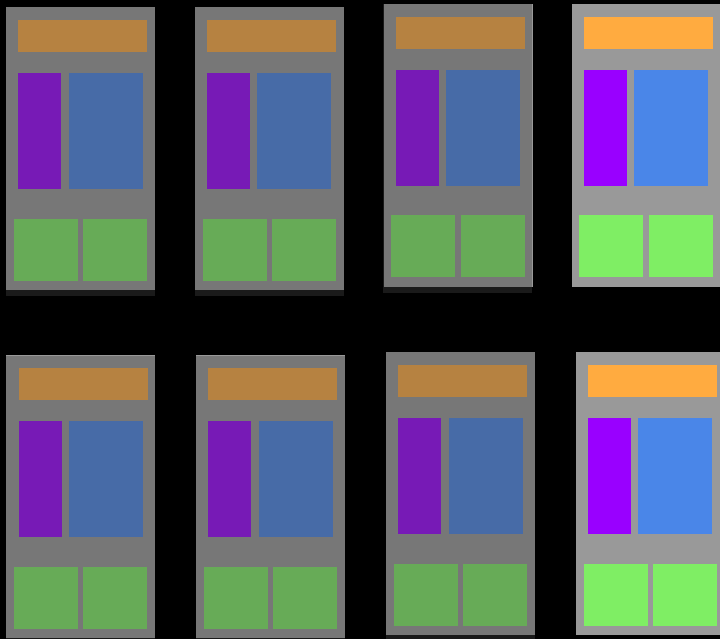
A machine is allocated
and an OS is booted.



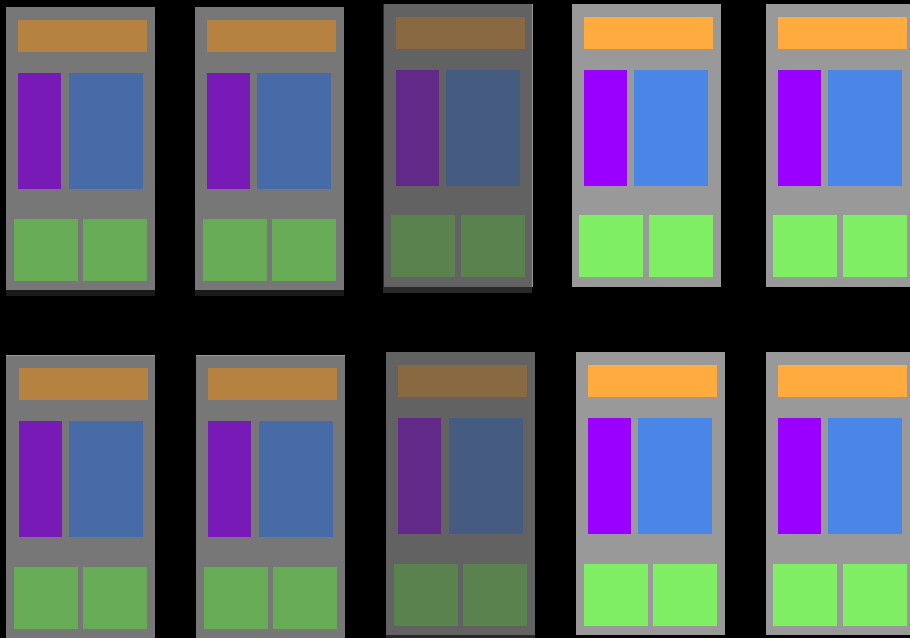
Remove instance start from the hot path with a buffer.



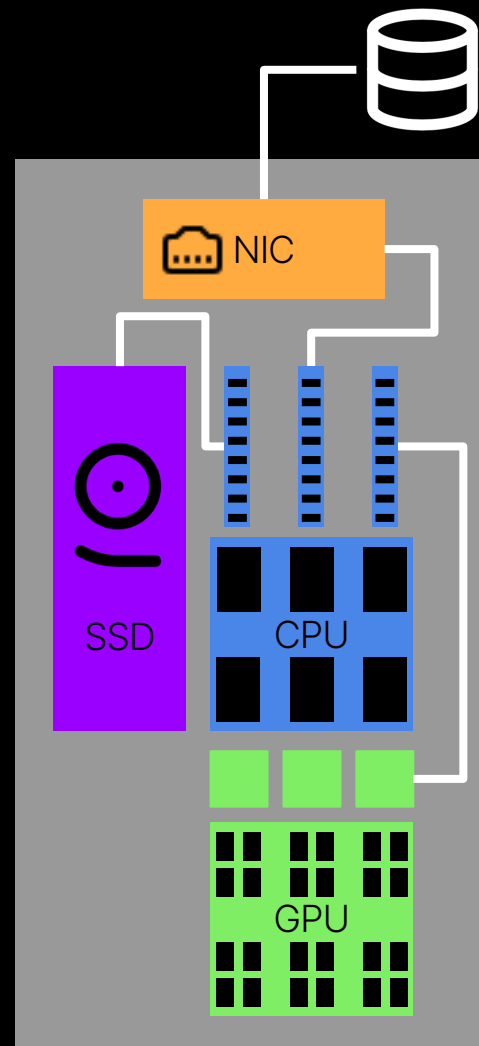
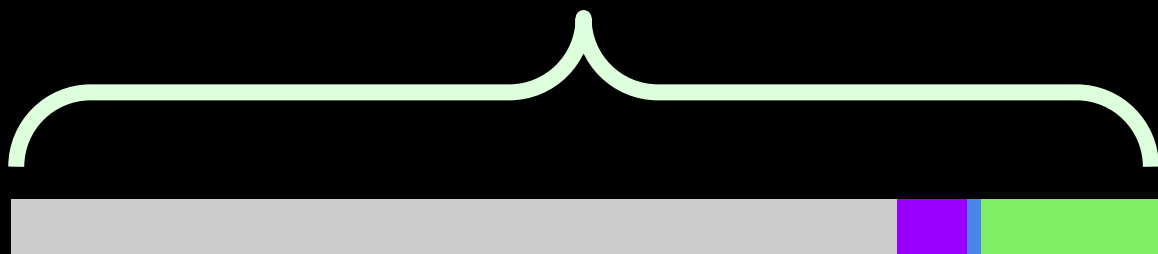
Remove instance start from the hot path with a buffer.



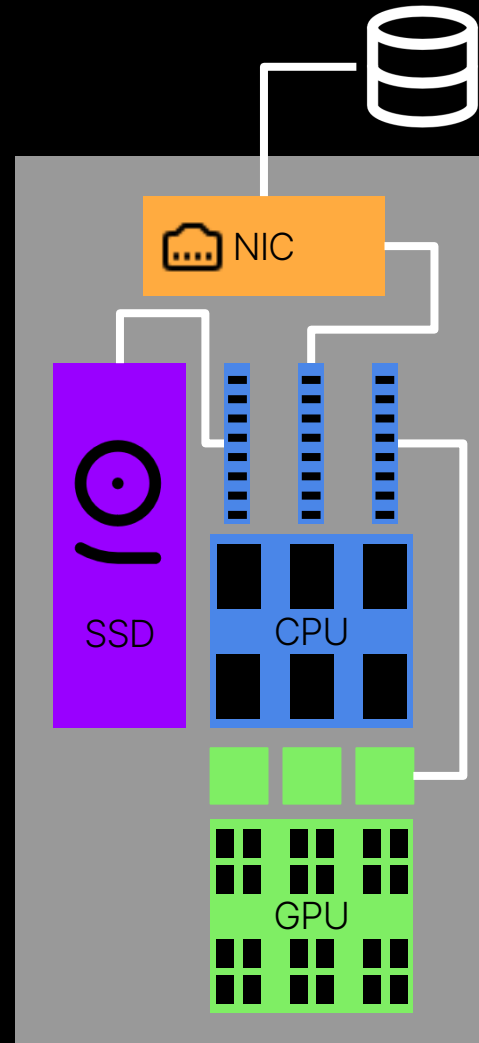
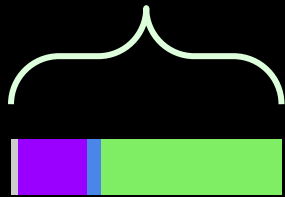
Remove instance start from the hot path with a buffer.



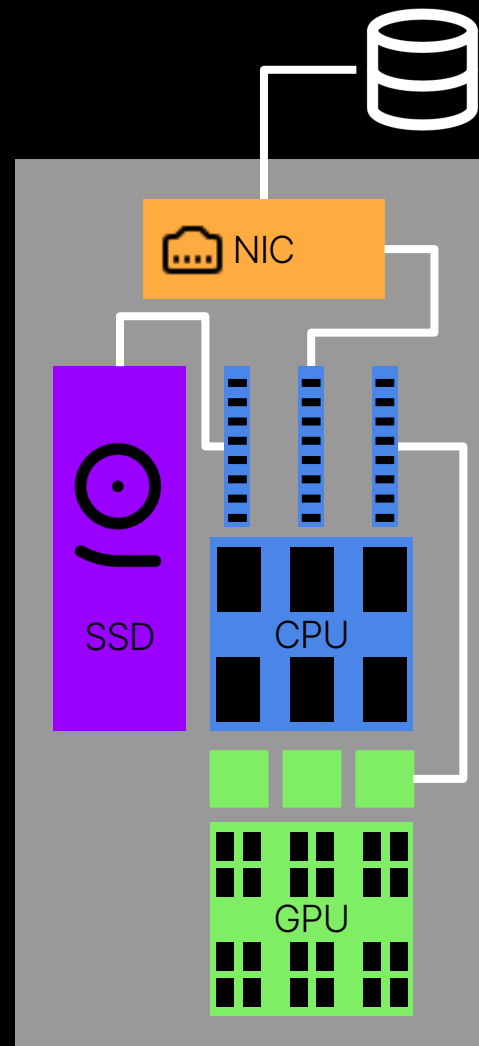
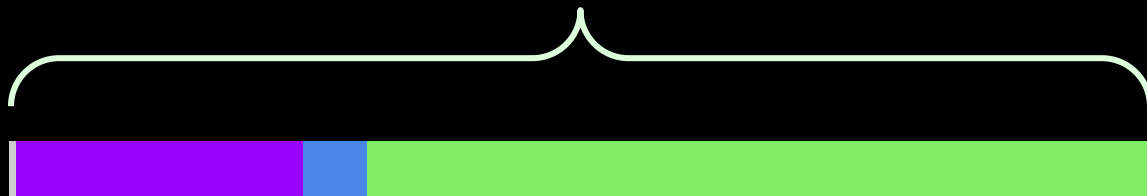
~2 ksec



~ 0.3 ksec

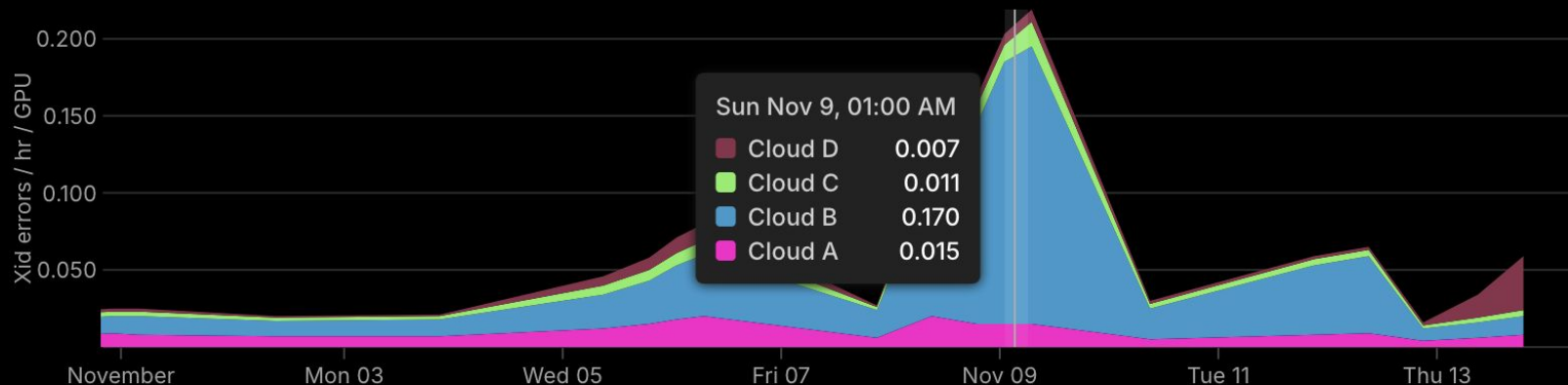


~ 300 s



Key component: health checks.

Passive healthchecking



Critical level Xid errors per hour by cloud, normalized by GPU count. Cloud B (blue) has by far the highest critical error rate.

Keeping 20,000 GPUs healthy



Jonathon Belotti @jonobelotti_IO
Member of Technical Staff



Amy Chang
Software Engineer Intern

Spin up a new instance

Eagerly load filesystem

Start application on host

Start application on device

Maintain a buffer

Lazy-load filesystem

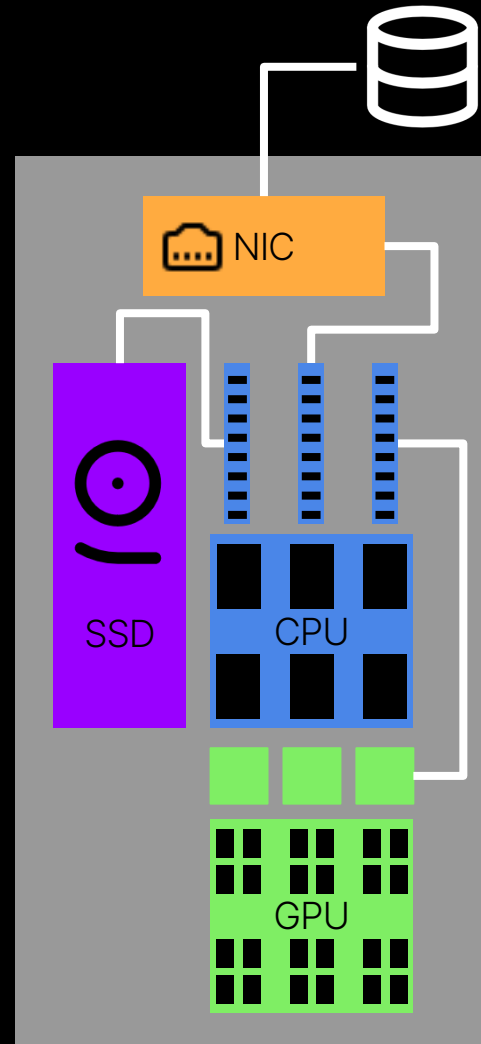
Restore Linux process

Restore CUDA ctx

What happens *here*?



A container is started up
inside the allocated machine.



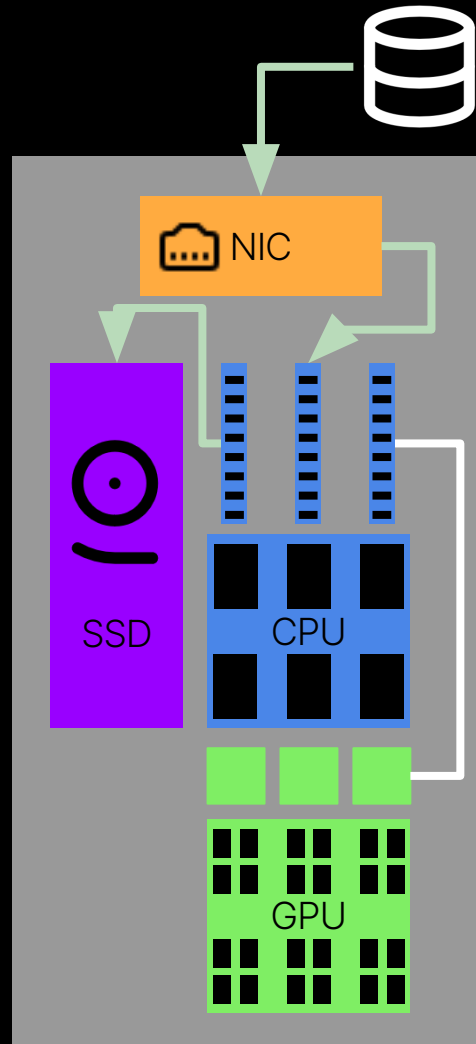
A container is a root FS that
you can run inside another root FS.

(plus a permissions/resources model)

Traditional container runtimes load this filesystem *eagerly*, layer by layer.



Need to load and decompress multiple GiB (~100 MiB/s).



Most files in container images are not read.

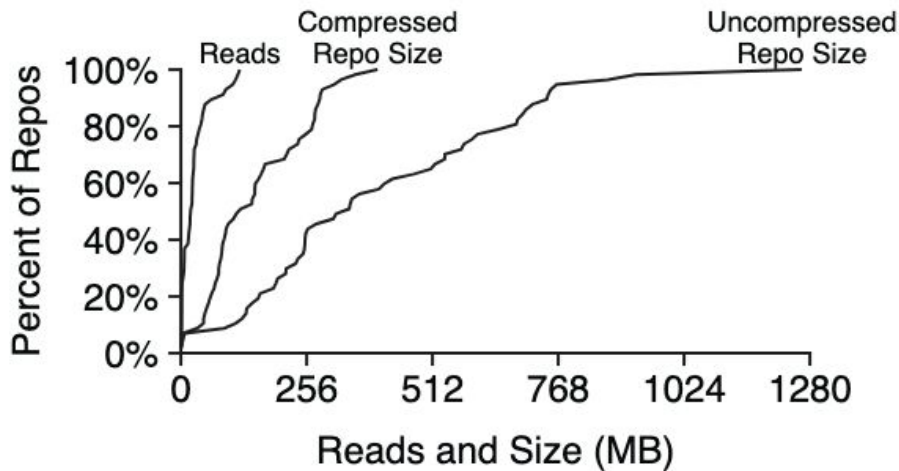
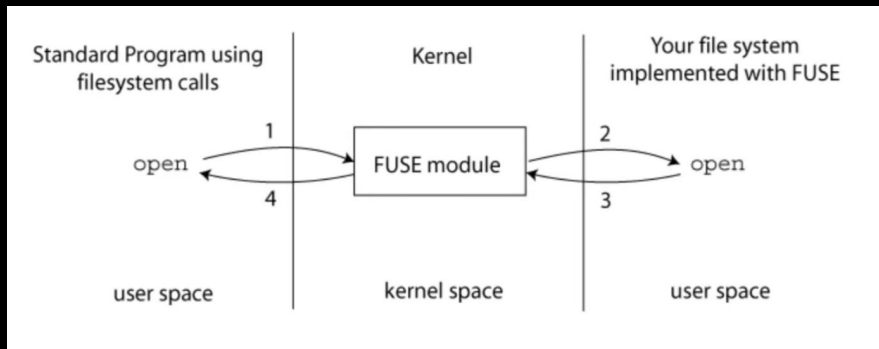
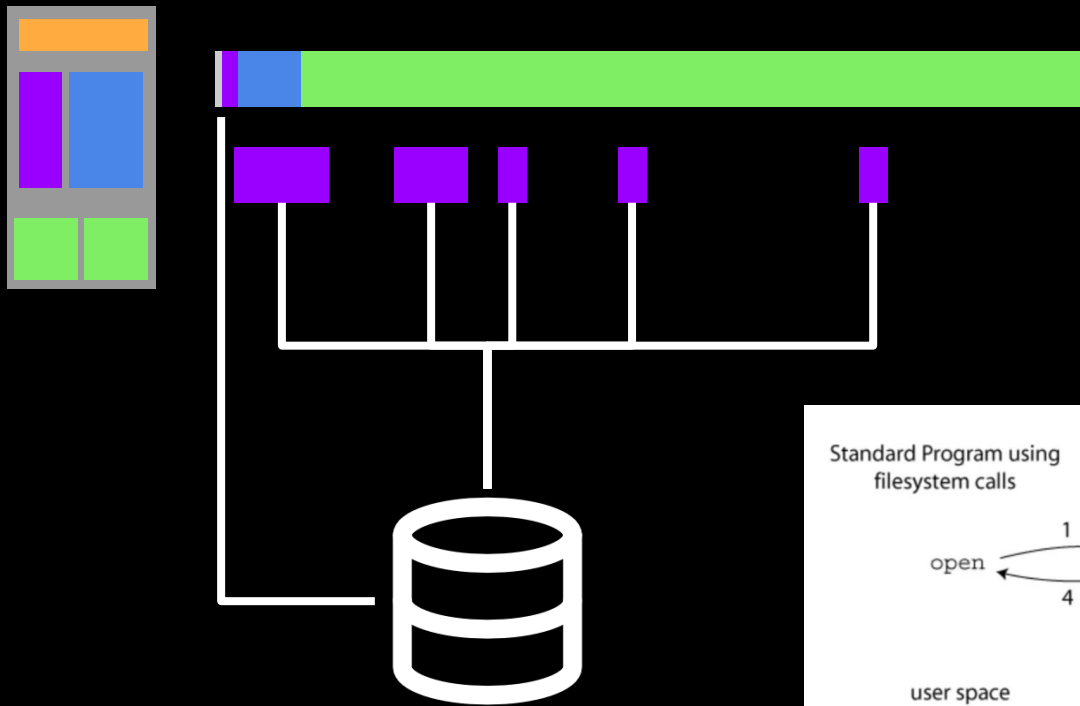
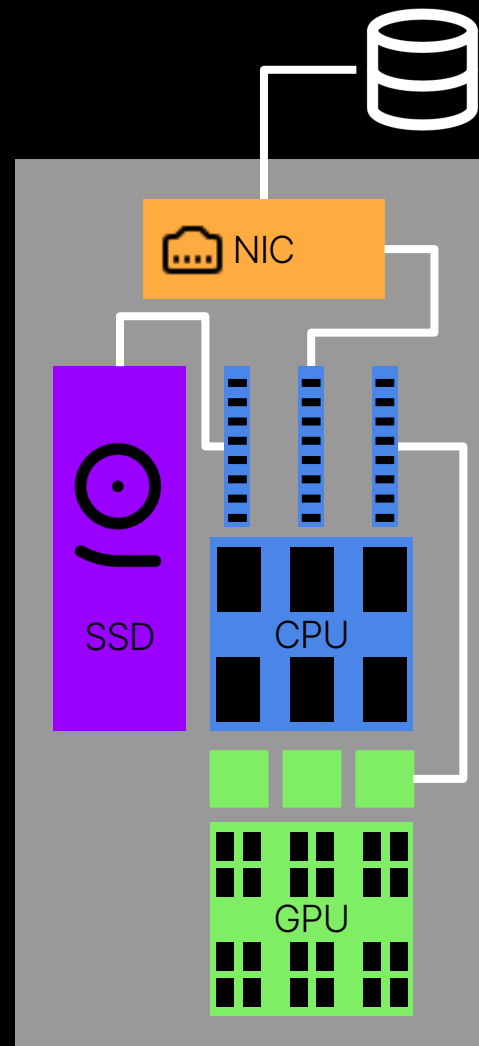
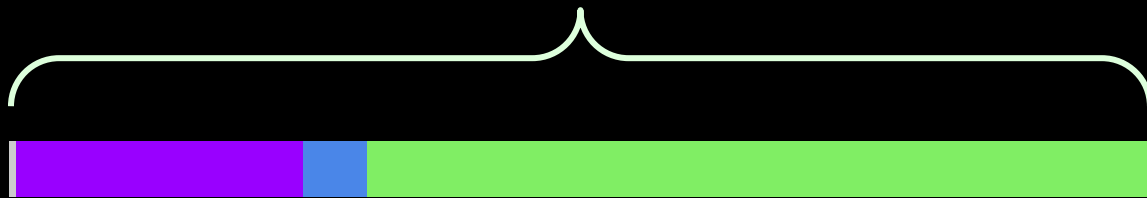


Figure 5: **Data Sizes (CDF).** *Distributions are shown for the number of reads in the HelloBench workloads and for the uncompressed and compressed sizes of the HelloBench images.*

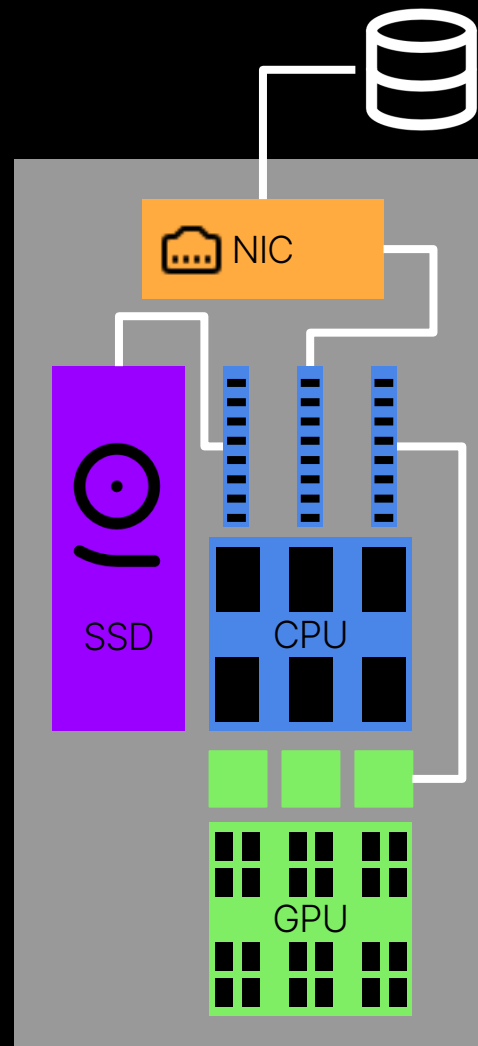
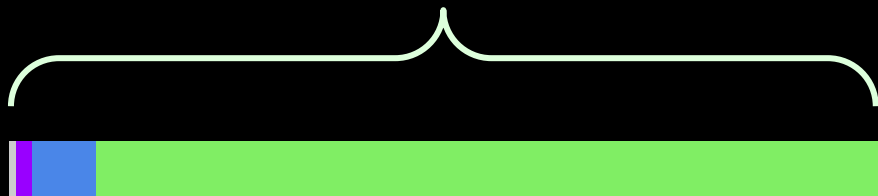
We wrote a custom FUSE file-system to load images async.



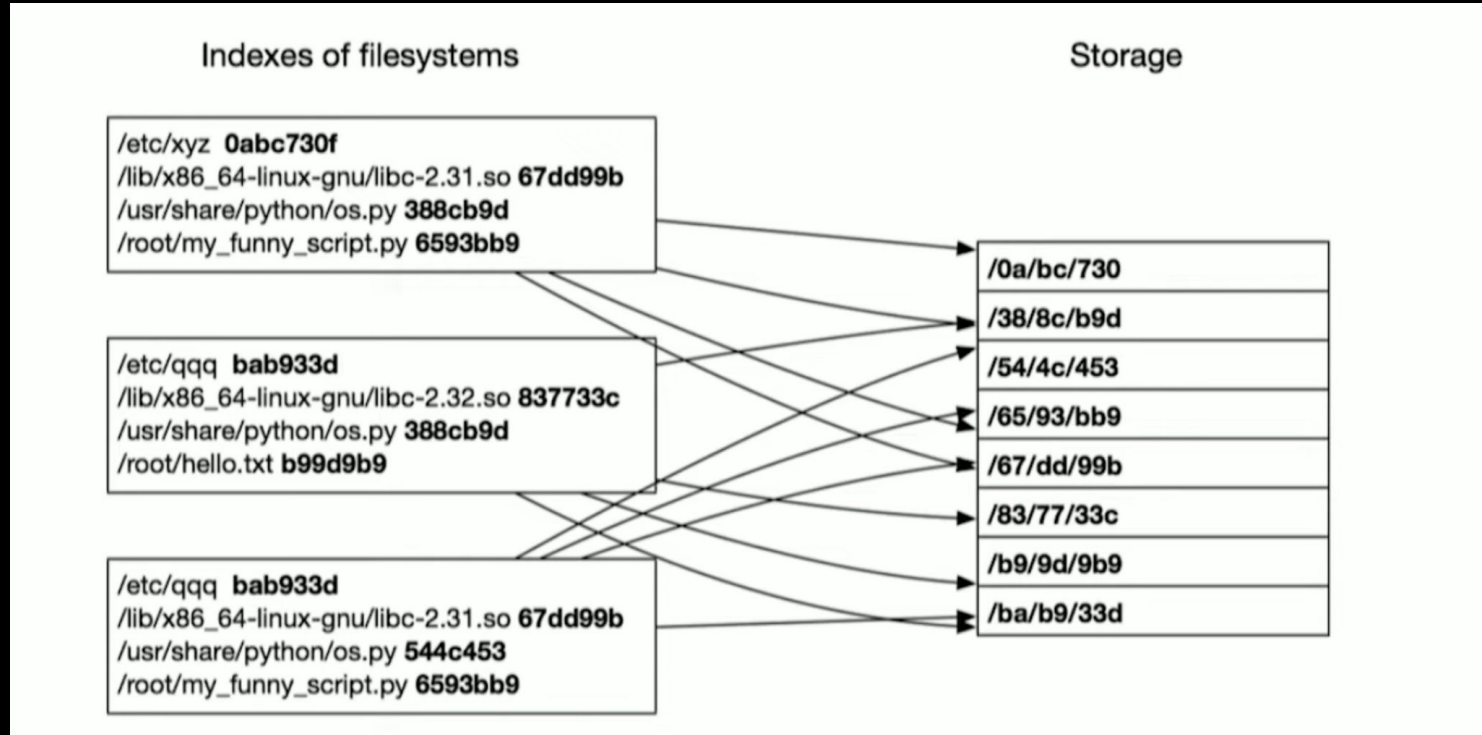
300 s



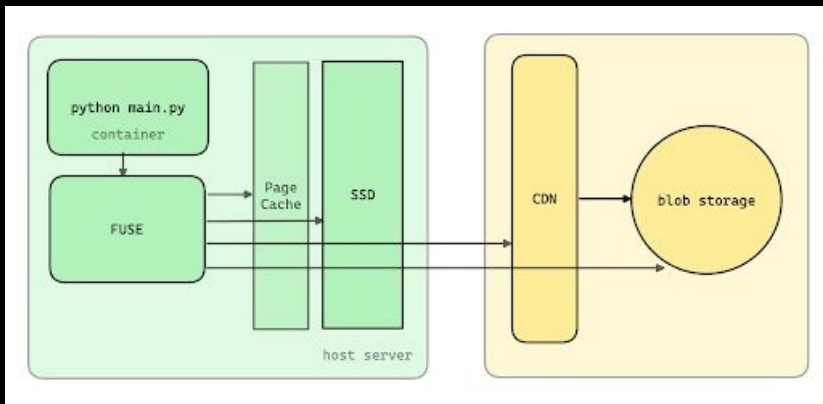
$\sim 200\text{ s}$



Key perf component: content-addressed cache

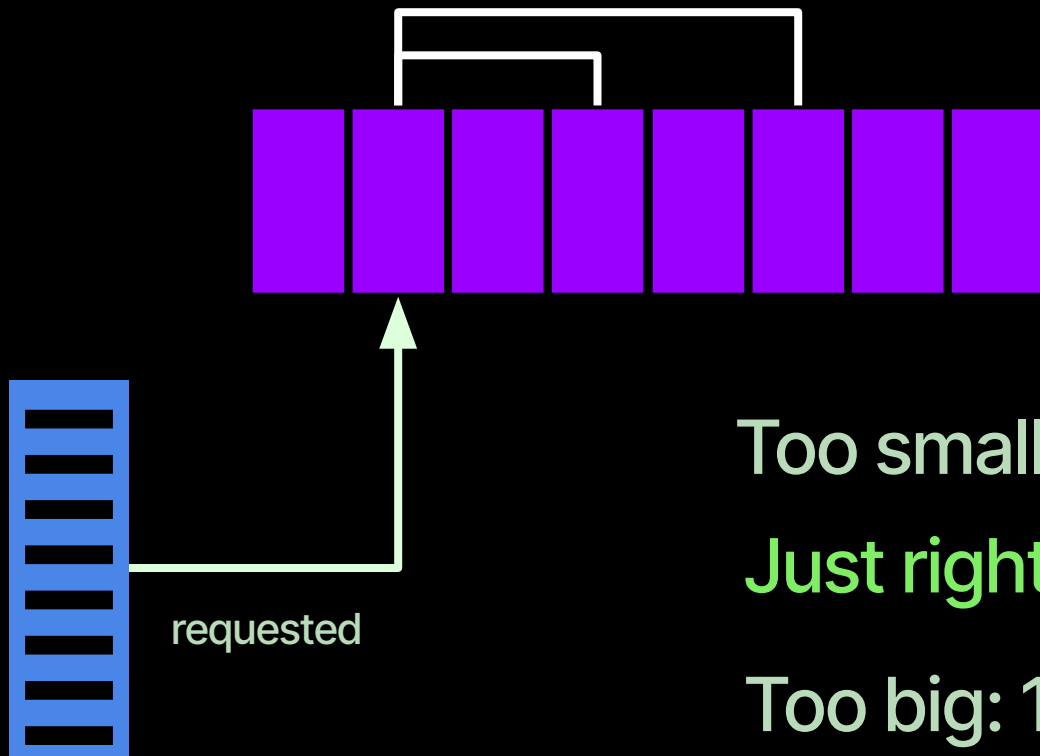


Key perf component: tiered "cloud-native" cache



System	Read Latency	Read Throughput	Cost (\$USD/GiB/month)
Memory	1-100 ns	10-40 GiB/s	\$2.0
SSD	100 μ s	4 GiB/s	\$0.10
AZ Cache Server	1 ms	10 GiB/s	\$0.15*
Regional CDN	100 ms	3-10 GiB/s	-
Blob Storage (e.g. S3)	200 ms	3-10 GiB/s	\$0.015

Key perf detail: libfuse readahead tuning



Too small: 128 KiB (default)

Just right: 32 MiB

Too big: 1 GiB (yikes)

Key perf detail: skip gzip!

System	Read Latency	Read Throughput	Cost (\$USD/GiB/month)
Memory	1-100 ns	10-40 GiB/s	\$2.0
SSD	100 µs	4 GiB/s	\$0.10
AZ Cache Server	1 ms	10 GiB/s	\$0.15*
Regional CDN	100 ms	3-10 GiB/s	-
Blob Storage (e.g. S3)	200 ms	3-10 GiB/s	\$0.015

gzip is single-threaded courtesy of
DEFLATE (LZ77, Huffman)

hard to beat 100 MiB/s,
10x slower than cloud storage

cloud storage is cheap

Spin up a new instance

Eagerly load filesystem

Start application on host

Start application on device

Maintain a buffer

Lazy-load filesystem

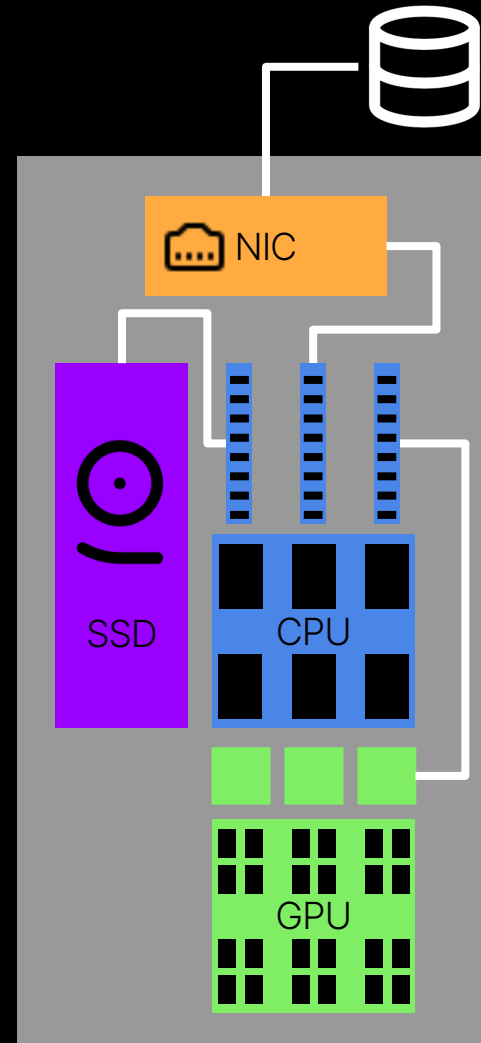
Restore Linux process

Restore CUDA ctx

What happens *here*?

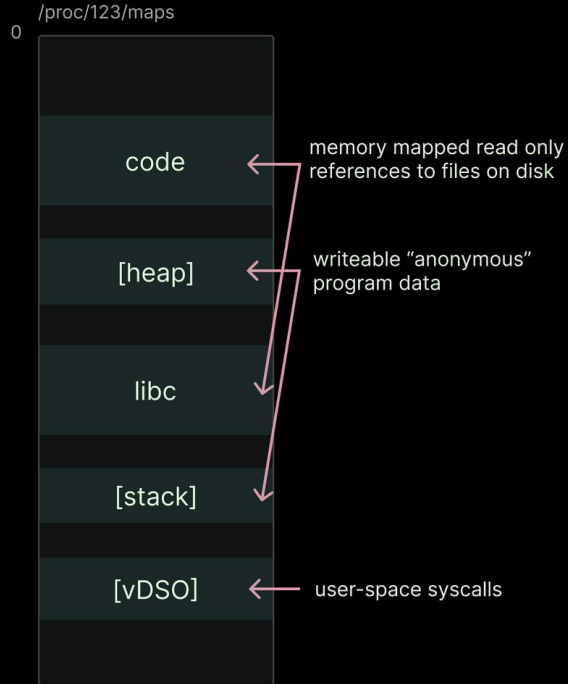


User code creates processes inside the container.



A process is a stack, a heap, registers, and kernel data.

Memory Mappings



Threads

main thread

```
tid = 123  
pid = 123  
registers:  
rip = 0x1234  
rsp = 0x2345  
rax = 0x9873  
rdi = 0x2834  
...
```

← thread id

← process id

← instruction pointer

← stack pointer

← general purpose registers

thread 2

```
tid = 243  
pid = 123  
...
```

File Descriptor Table

```
0 = stdin  
1 = stdout  
2 = stderr  
5 = myfile.txt  
6 = <tcp:88>  
7 = <tcp conn>
```

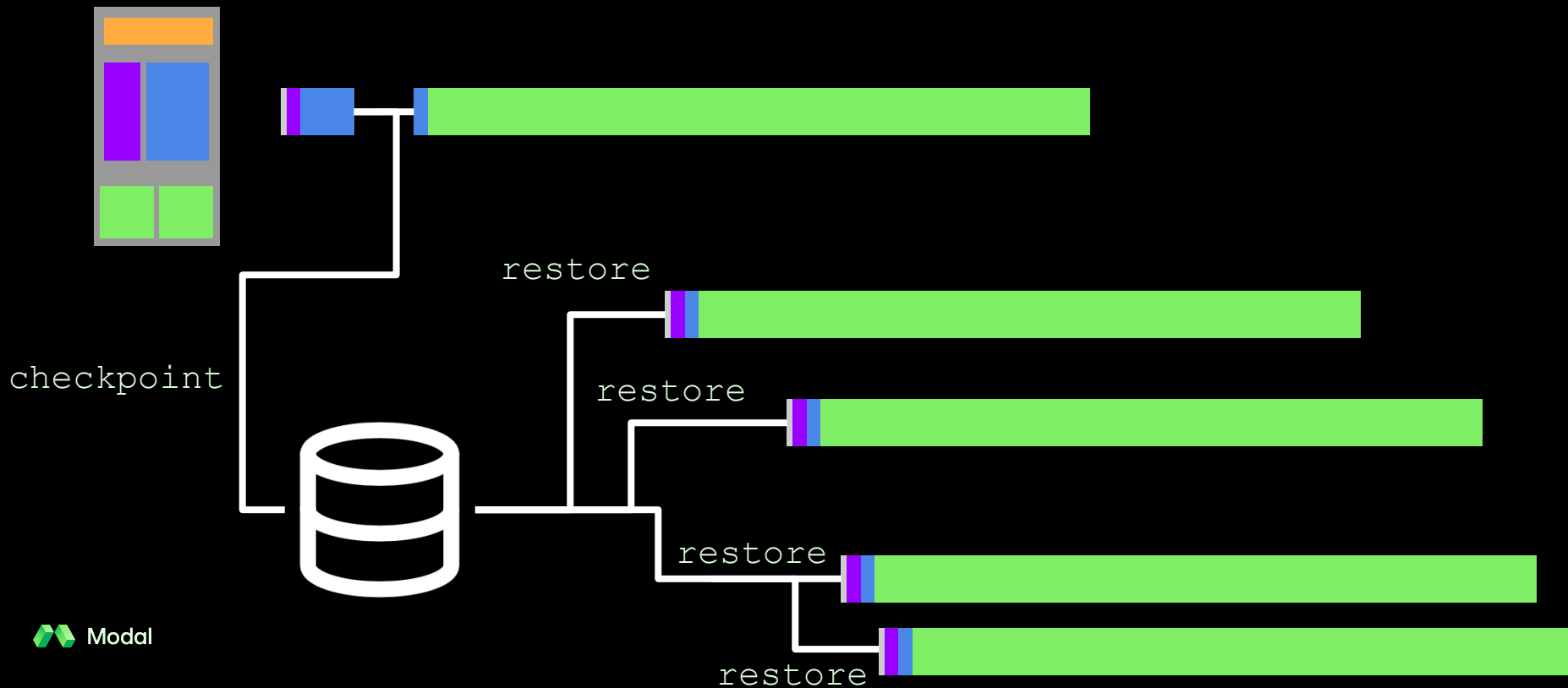
← standard channels (0, 1, 2)

← open file

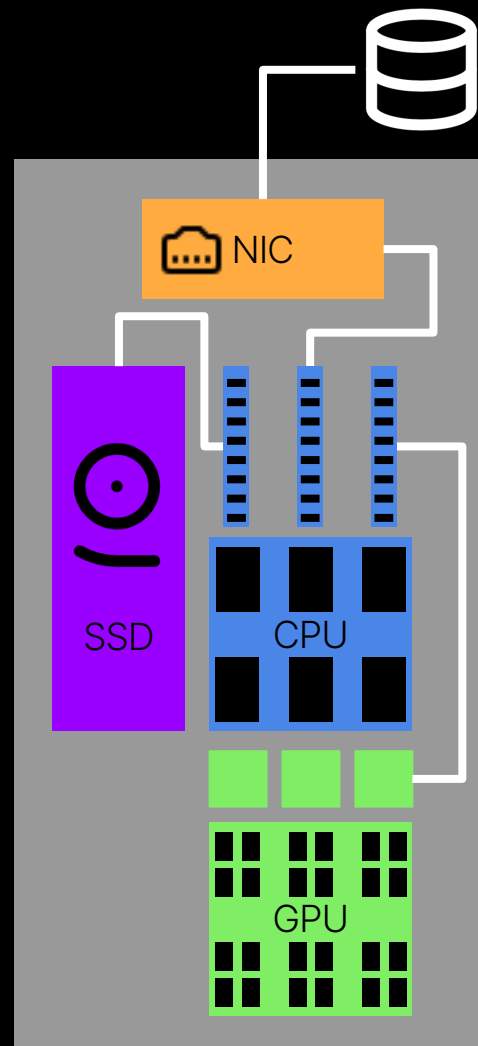
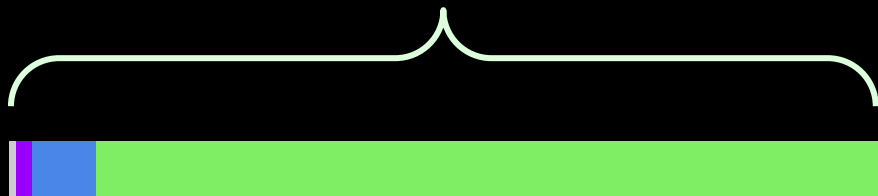
← tcp listening socket

← socket to tcp client

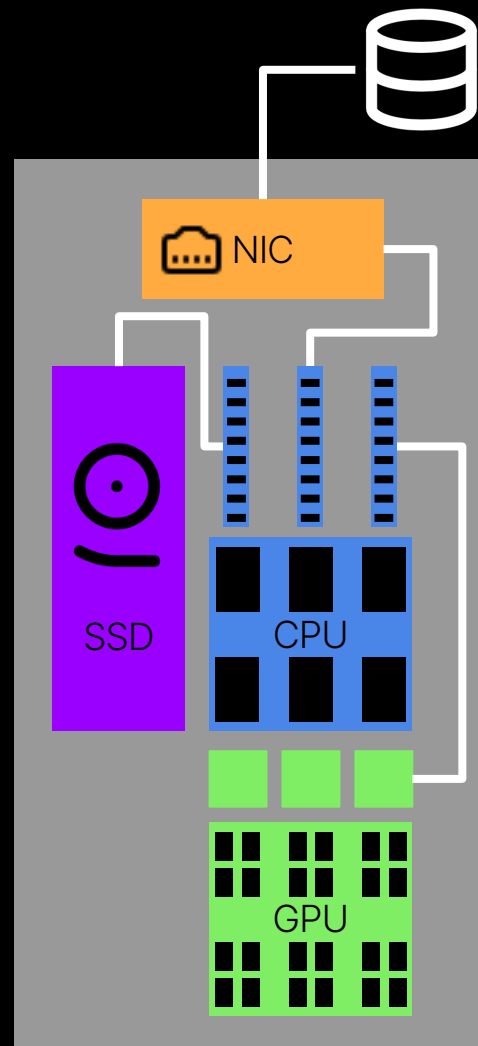
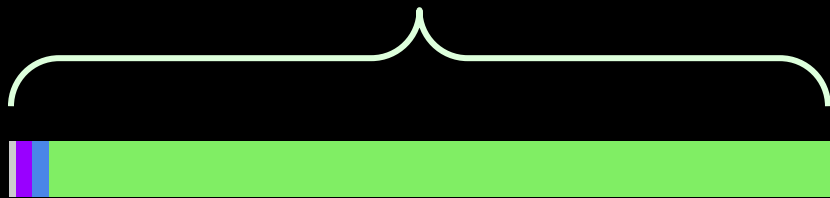
Store that data so we can skip the work next time.



$\sim 200\text{ s}$



~ 200 s



For fast startups, only need checkpointing at startup.

```
@app.cls(enable_memory_snapshot=True) # checkpoint global scope
class InferenceService:
    @modal.enter(snap=True) # checkpoint when this method returns
    def startup(self):
        ...

    @modal.enter(snap=False) # run in container init, but don't check
    def finalize(self):
        ...

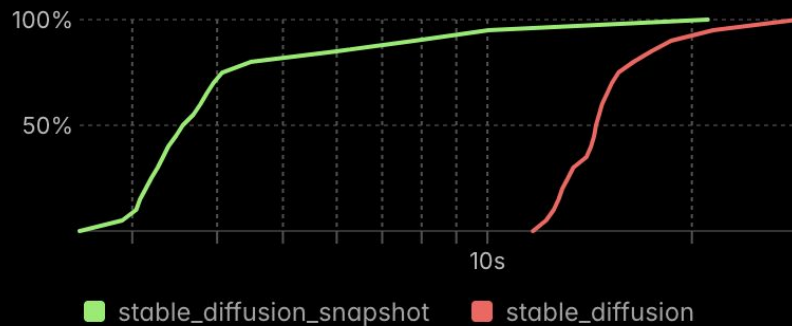
    @modal.method() # run with each invocation
    def run(self, messages):
        ...
```

Key perf win: Python imports.

import torch



Stable Diffusion



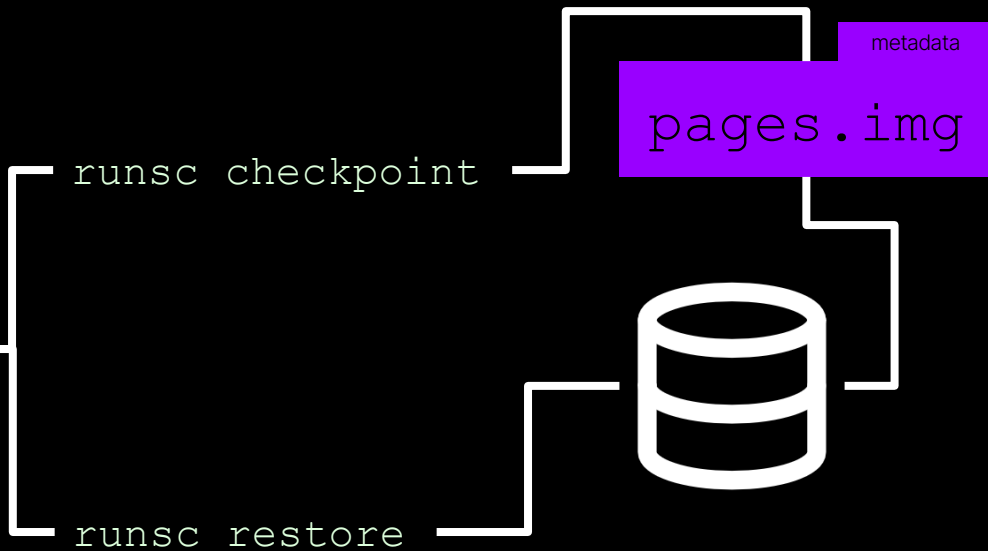
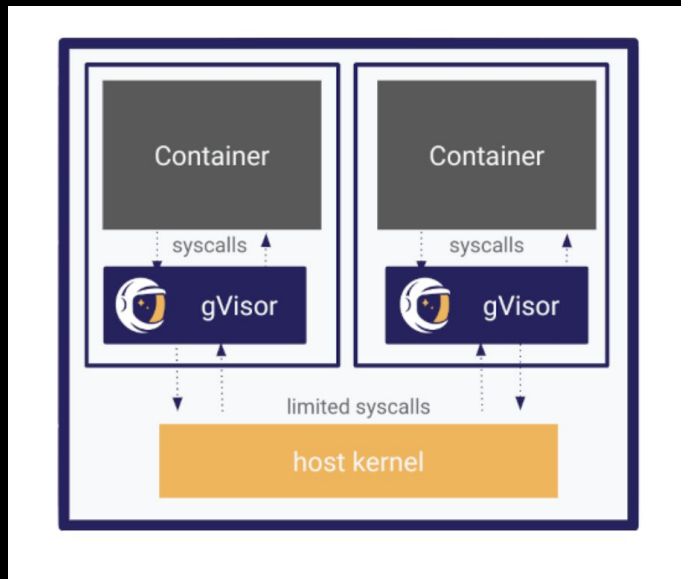
Memory Snapshots: Checkpoint/Restore for Sub-second Startup



Jonathon Belotti @jonobelotti_IO
Member of Technical Staff

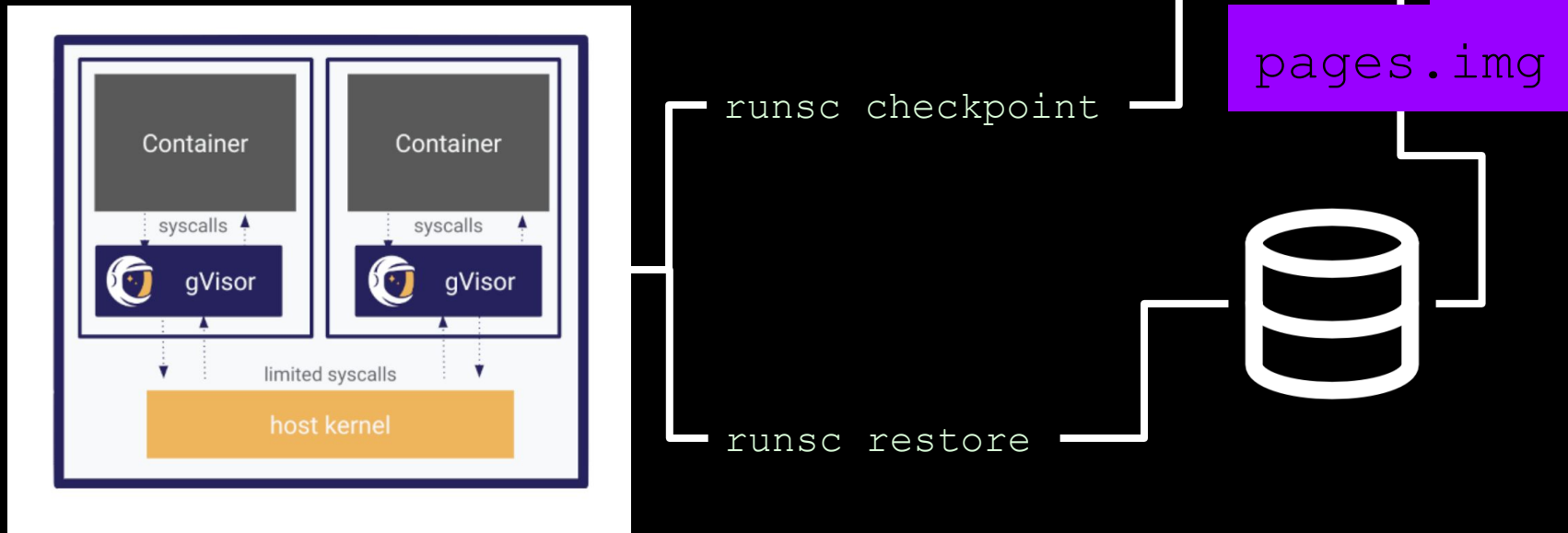
<https://modal.com/blog/mem-snapshots>

We use gVisor Checkpoint/Restore.



Containers in gVisor are state machines,
so checkpoint/restore is "easy".

Key perf target: restoring `pages.img`.



Spin up a new instance

Eagerly load filesystem

Start application on host

Start application on device

Maintain a buffer

Lazy-load filesystem

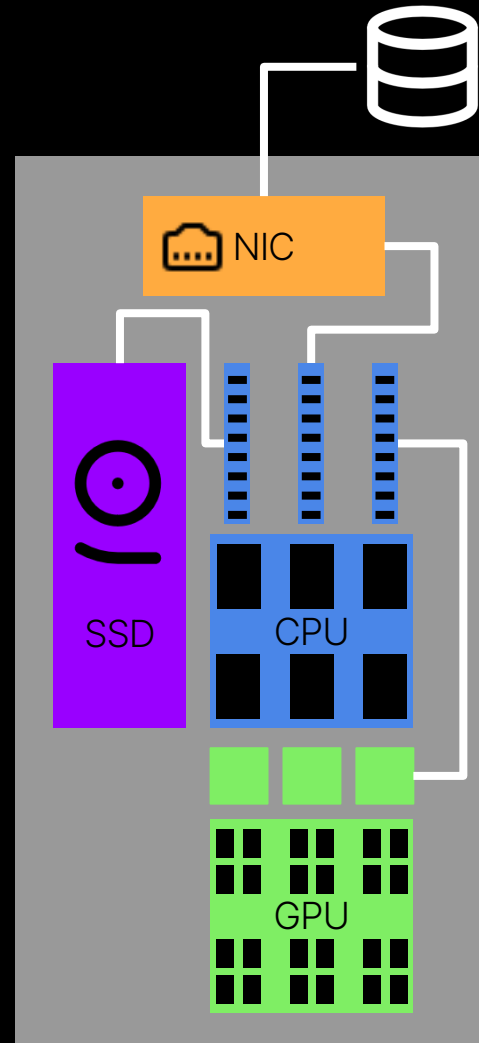
Restore Linux process

Restore CUDA ctx

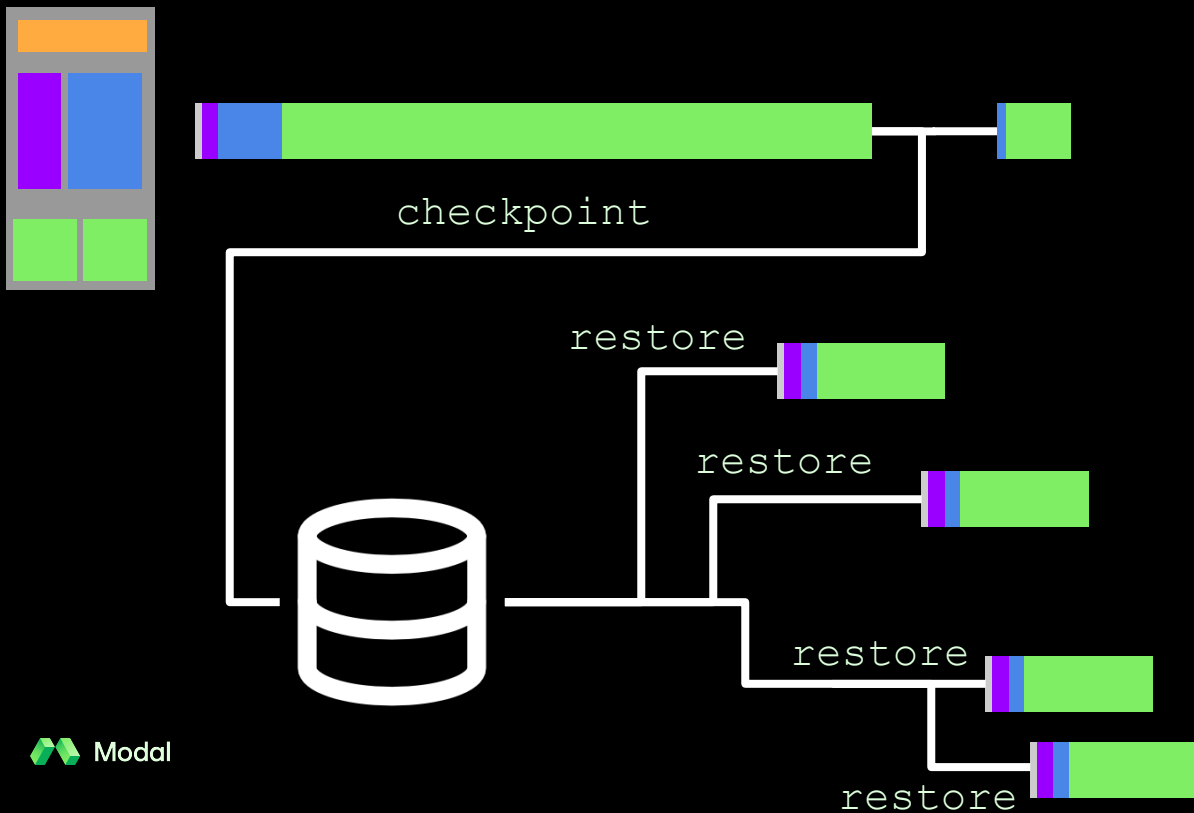
What happens *here*?

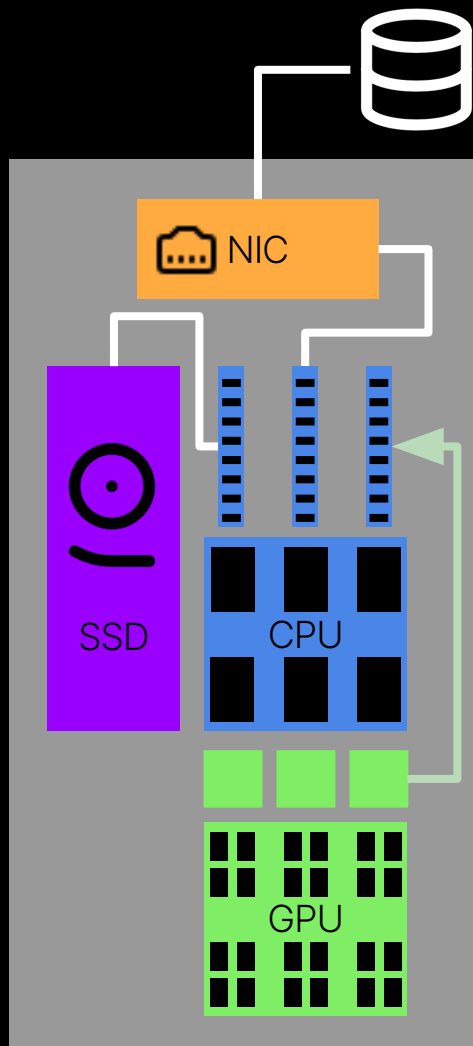


User code creates CUDA resources inside a context on the device.

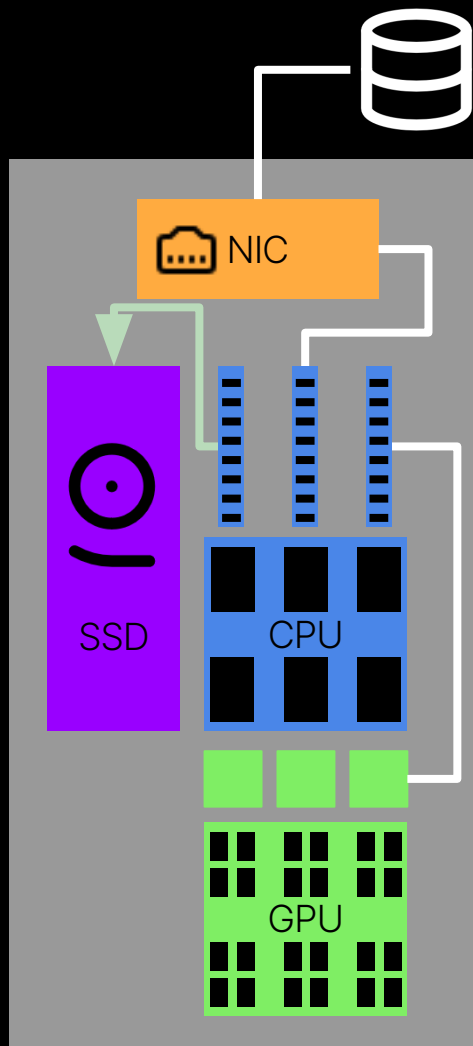


Store that data so we can skip the work next time.

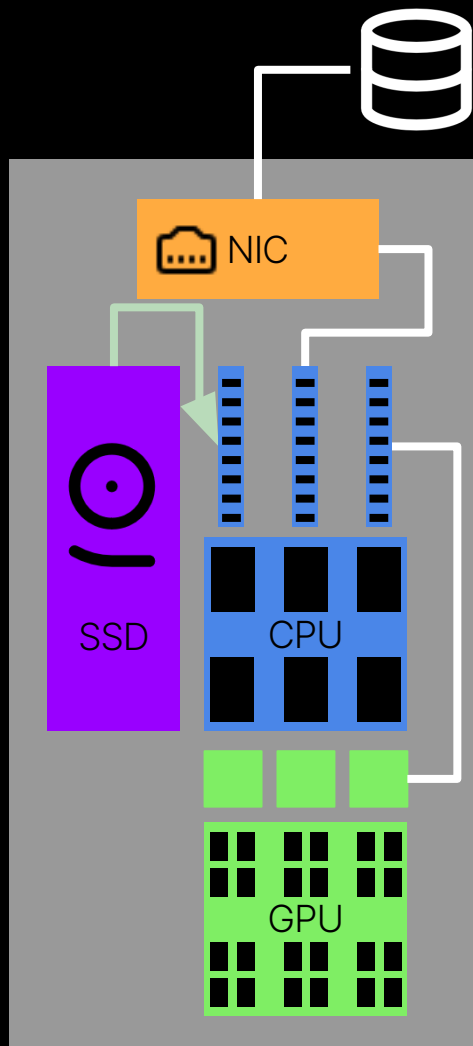




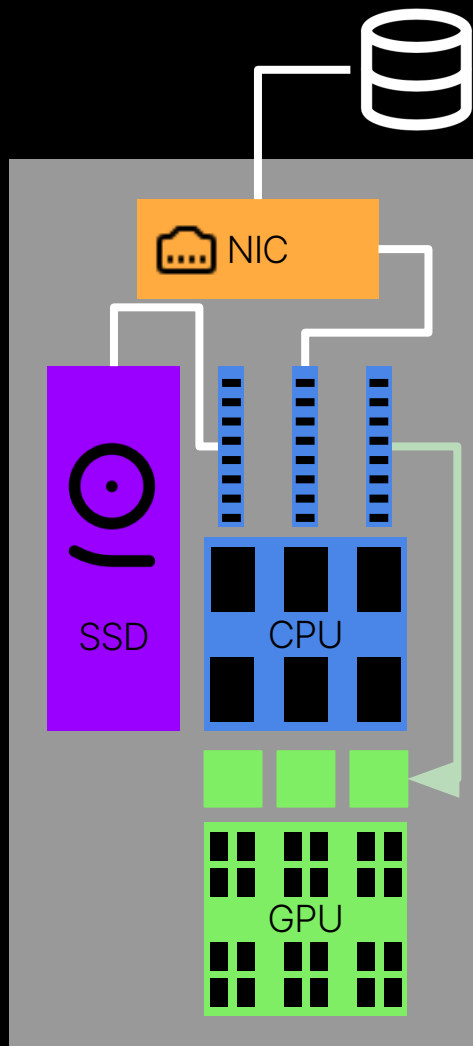
```
cuda-checkpoint --toggle
```



`runsc checkpoint`

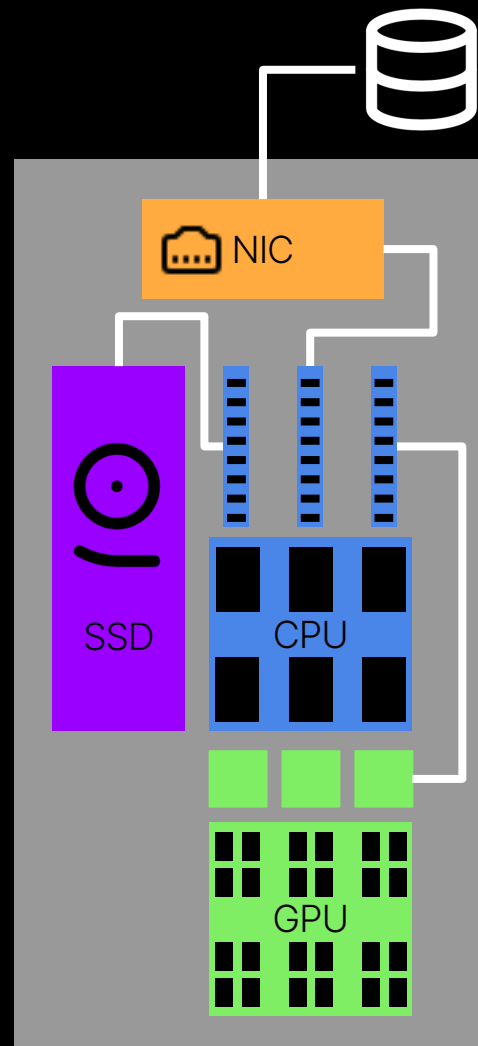
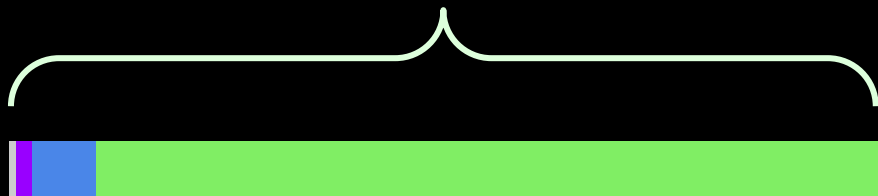


`runsc restore`

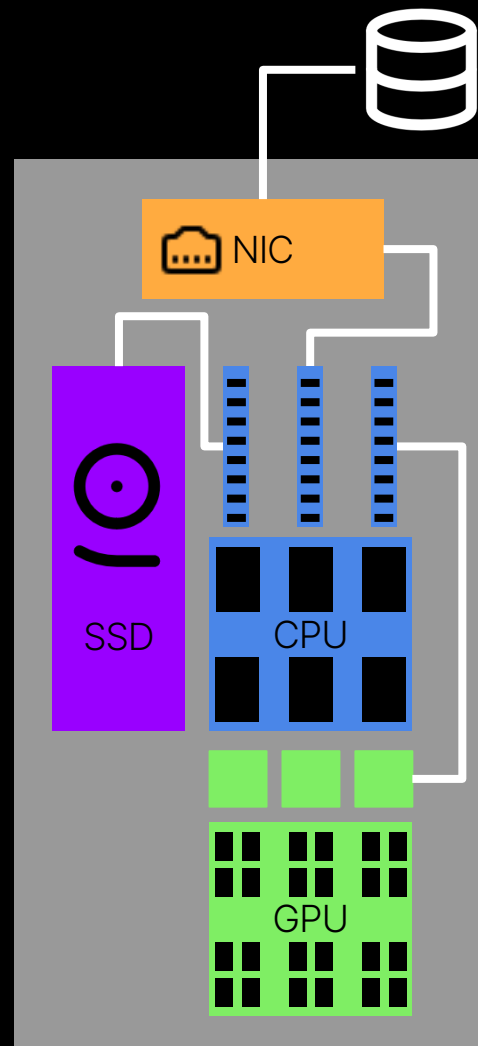
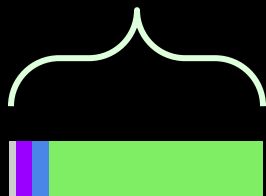


```
cuda-checkpoint --toggle
```

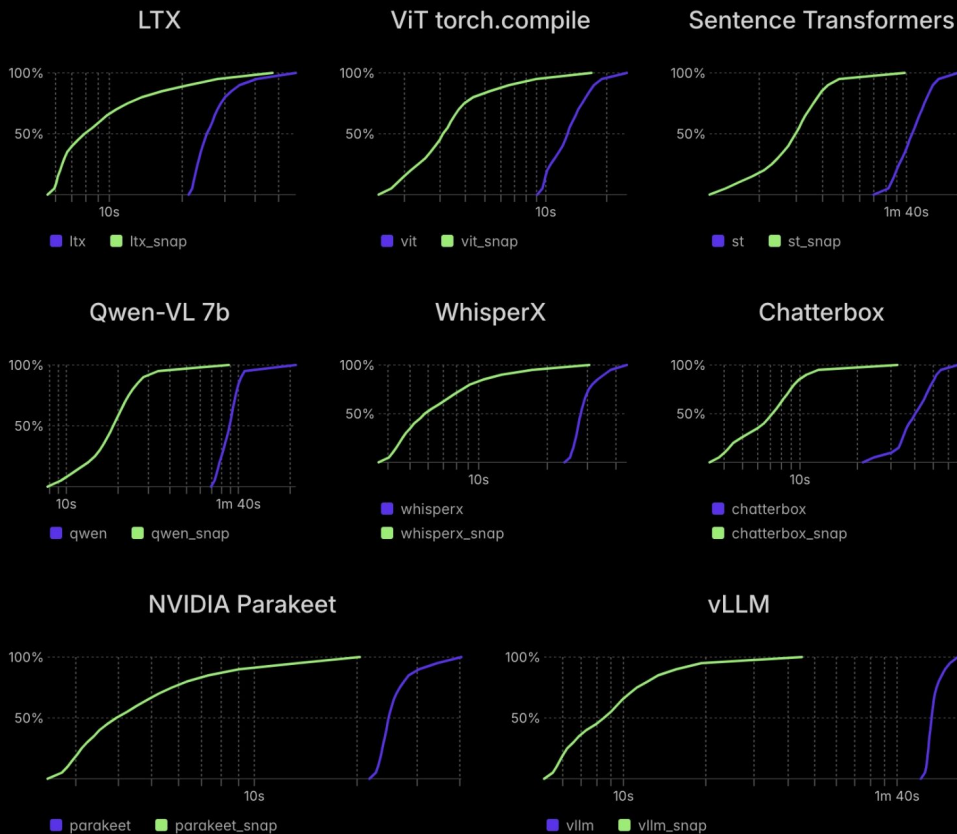
$\sim 200\text{ s}$



~30 s



Benchmark results: 3-10x speedups.



Key perf considerations

Can skip "JIT compilation"

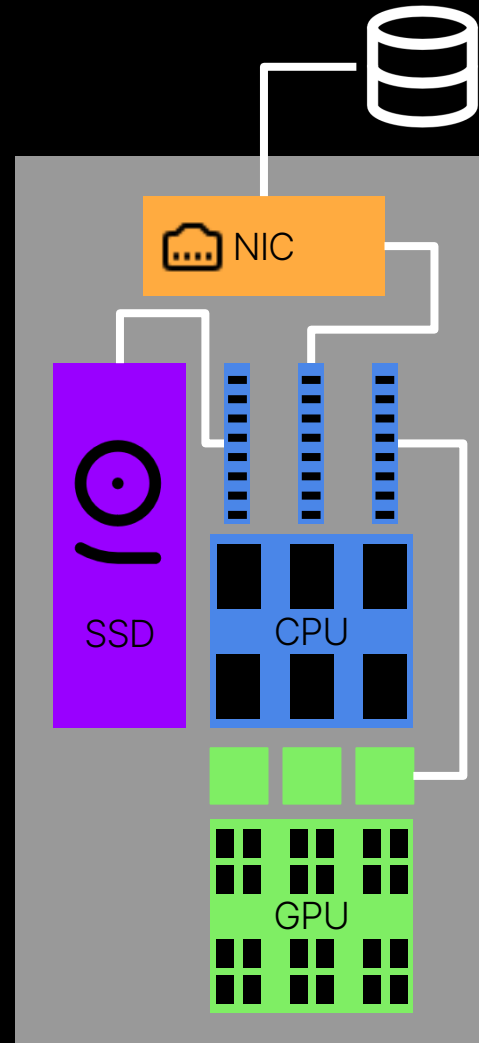
- CUDA Graph capture, Torch Compiler(s)

Pro tip: dump the (cold) KV cache, you can recreate it faster than you can load it.

What happens *here*?



Model weight loading, mostly.
Solve with more caches? RDMA?
Hard to do globally,
but we're experimenting.



Checkpoint and restoring CUDA applications in practice.

~12
Months GA

*running CPU
Snapshots*

~ $10^{6.5}$
CPU
Processes

restored

~ $10^{4.5}$
Snapshots

created

~ 10^6
Wall Hours

*executed
after restore*

~ $10^{2.5}$
Organizations

*using
CPU Snapshots*

~6
Months GA

*running GPU
Snapshots*

~ 10^6
GPU
Contexts

restored

~ 10^4
GPU
Snapshots

created

~ 10^5
GPU-Hours

*executed
after restore*

~ $10^{2.5}$
Organizations

*using
GPU Snapshots*

Use cases

Custom filesystem

Cloud-native development, Notebooks

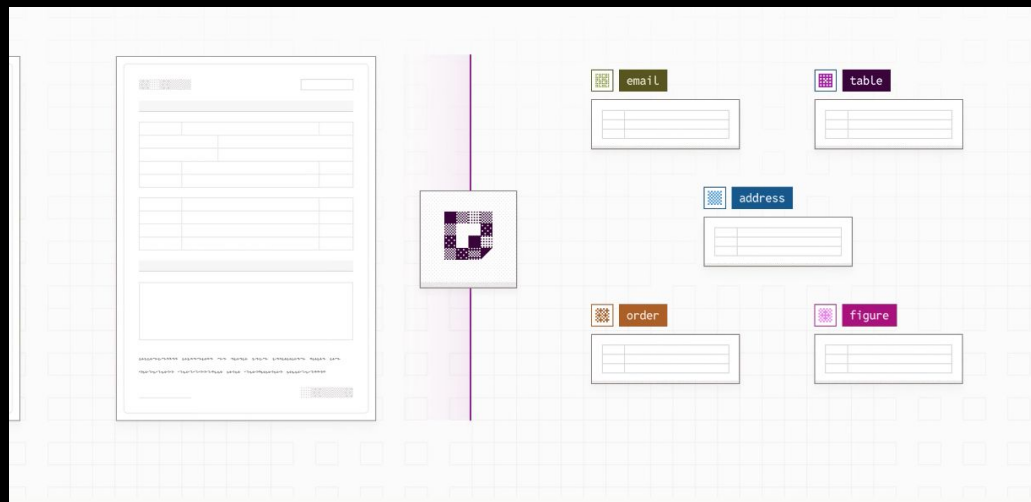
CPU Snapshots

Data processing pipelines

GPU Snapshots

“Not Large” models (OCR, STT/TTS, medium LMs)

Case study: Enterprise doc processing by Reducto



70%

*reduction
in cold start time*

10^3

*GPUs allocated in a
single short burst*

$\sim 10^6$

*PDFs, spreadsheets,
slide decks per day*

Do you want to **build this yourself?**

Come talk to us! Booth #4020

Do you want to **build this with us?**

<https://modal.jobs>

Do you want to **try it?**

<https://modal.com>



Modal